

Разработка инфраструктуры бенчмаркинга графовых алгоритмов поиска пути с ограничениями

Глазунов Иван Сергеевич

Группа: 23.Б10-мм

Кафедра: Системного программирования

Научный руководитель: доцент кафедры системного программирования, к. ф.-м. н. Григорьев Семён Вячеславович

Консультант: Белянин Георгий Олегович. ООО «ВК Цифровые Технологии», разработчик

Номер семестра практики: 5

1. Постановка задачи

Работа выполняется в рамках научно-исследовательского проекта по исследованию алгоритмов исполнения запросов на поиск путей в графе с ограничениями в терминах формальных языков. **Целью работы является** разработка и реализация инфраструктуры для запуска, тестирования и сравнения алгоритмов поиска путей с ограничениями, а также их оптимизаций.

Результаты работы направлены на стандартизацию и упрощение процесса сравнения производительности разрабатываемых алгоритмов [2, 5]. Конечными пользователями такой инфраструктуры являются исследователи и разработчики графовых алгоритмов, которым требуется воспроизводимо запускать эксперименты на общих наборах данных и сопоставимых настройках.

2. Описание предлагаемого решения

На текущий момент различие в интерфейсах алгоритмов поиска путей с ограничениями затрудняет сравнение различных подходов. Разные алгоритмы оперируют с отличающимися форматами графов, определяют запросы при помощи различных абстракций. Не хватает единого решения, которое позволяло бы работать с разными входными и выходными форматами графовых алгоритмов единым образом.

Предлагаемое решение заключается в разработке библиотеки и консольной утилиты для выполнения запросов с ограничениями на пути и запуска воспроизводимых бенчмарков. В рамках работы реализована поддержка нескольких форматов входных графов, разбор запросов на подмножестве SPARQL [7], единый интерфейс для подключения алгоритмов, сбор статистики запусков и восстановление прогресса после прерывания бенчмарка.

С архитектурой решения можно ознакомиться в репозитории проекта. Реализация выполнена на языке Rust; для вычислений над разреженными булевыми матрицами используются SuiteSparse:GraphBLAS [8] и алгоритмы библиотеки LAGraph [4]. Сборка проекта автоматически подготавливает нативные зависимости: GraphBLAS и LAGraph собираются статически, поэтому пользователю не требуется устанавливать их вручную. Проект также подготовлен к распространению как Rust-библиотека через crates.io и снабжён автоматически публикуемой документацией на docs.rs.

3. Эксперименты

Целью эксперимента была оценка накладных расходов при запуске LARPQ через библиотеку и CLI Pathrex по сравнению с запуском исходной реализации, а также сравнение с MillenniumDB [6].

Эксперименты проводились на реальном наборе данных Wikidata [9]. Датасет разделен на простые и сложные запросы.

В таблице приведены средние времена исполнения запросов в секундах и относительное замедление по сравнению с LARPQ (меньше — лучше) для простых и сложных (C) запросов отдельно.

Таблица 1: Результаты измерений на наборе данных Wikidata.

Измерение	LARPQ	Pathrex	MillenniumDB
Mean	0.016	0.020	0.076
Mean slowdown	1.00	1.30	4.85
Mean (C)	1.691	2.093	2.886
Mean slowdown (C)	1.00	1.24	1.71

На простых запросах Pathrex работает в 4 раза быстрее MillenniumDB при накладных расходах около 30% относительно исходной реализации LARPQ. На сложных запросах Pathrex работает в 1.4 раза быстрее MillenniumDB при накладных расходах около 24% относительно LARPQ. Таким образом, наличие дополнительной инфраструктуры для алгоритмов поиска путей на основе линейной алгебры не препятствует их высокой производительности. Подробнее ознакомиться с экспериментами можно в репозитории <https://github.com/VanyaGlazunov/pathrex-bench>.

4. Заключение

В ходе выполнения работы получены следующие результаты:

- Спроектирована архитектура библиотеки и утилиты для тестирования алгоритмов поиска путей с ограничениями
- Поддержаны форматы входного графа CSV, MatrixMarket и RDF, а также разбор регулярных запросов на основе SPARQL
- Встроены два алгоритма для регулярных запросов: LARPQ [2] и RPQ-Matrix [1]
- Реализован запуск бенчмарков с сохранением подробной статистики и восстановлением после прерывания
- Добавлены unit-, интеграционные и end-to-end тесты для форматов данных, построения графа, CLI и встроенных алгоритмов
- Настроены публикация Docker-образов, распространение пакетов через crates.io и публикация документации на docs.rs

Репозиторий: <https://github.com/SparseLinearAlgebra/pathrex>.

Документация: <https://docs.rs/pathrex/latest/pathrex/>

Cargo-пакет: <https://crates.io/crates/pathrex>

Dockerhub: <https://hub.docker.com/repository/docker/vanyaglazunov/pathrex/general>

Решение находится в разработке. В дальнейшем планируется добавить алгоритмы для контекстно-свободных грамматик и поддержку оптимизаций при помощи E-Graph [3].

Список литературы

- [1] Arroyuelo Diego, Gómez-Brandón Adrián, Navarro Gonzalo. Evaluating Regular Path Queries on Compressed Adjacency Matrices // String Processing and Information Retrieval: 30th International Symposium, SPIRE 2023, Pisa, Italy, September 26–28, 2023, Proceedings. — Berlin, Heidelberg : Springer-Verlag, 2023. — P. 35–48. — URL: https://doi.org/10.1007/978-3-031-43980-3_4 (дата обращения: 2026-05-04).
- [2] Belyanin Georgiy, Grigoriev Semyon, Suvorov Rodion. Single-Source Regular Path Querying. — URL: <https://arxiv.org/abs/2412.10287> (дата обращения: 2026-05-04).
- [3] Tate Ross, Stepp Michael, Tatlock Zachary, Lerner Sorin. Equality Saturation: A New Approach to Optimization. — URL: <https://arxiv.org/abs/1012.1802> (дата обращения: 2026-05-04).
- [4] LAGraph. — URL: <https://github.com/graphblas/lagraph> (дата обращения: 2026-05-12).
- [5] LA'n'EGG. — URL: <https://github.com/SparseLinearAlgebra/la-n-egg-rpq> (дата обращения: 2026-05-12).
- [6] MillenniumDB repository. — URL: <https://github.com/MillenniumDB/MillenniumDB> (дата обращения: 2026-05-04).
- [7] SPARQL 1.1 Query Language : Rep. / W3C : 2013. — URL: <http://www.w3.org/TR/sparql11-query> (дата обращения: 2026-05-04).
- [8] SuiteSparse:GraphBLAS. — URL: <https://github.com/DrTimothyAldenDavis/GraphBLAS> (дата обращения: 2026-05-12).
- [9] Wikidata dataset. — URL: <https://github.com/MillenniumDB/path-query-challenge> (дата обращения: 2026-05-04).